

QUESTION 26

Write pseudocode for a real-time data acquisition system using timers, semaphores, ISR, and queues.

ANSWER

1. PROBLEM UNDERSTANDING

A real-time data acquisition system must collect sensor samples at a fixed interval, respond to a hardware interrupt, synchronize the acquisition operation, and pass completed samples safely to another task for processing.

2. RTOS MECHANISM USED

A periodic timer establishes the sampling schedule. A binary semaphore releases the acquisition task at the required interval. An ISR handles the hardware data-ready event with short interrupt work. A queue transfers completed samples from acquisition to processing without direct task coupling.

3. STRUCTURED PSEUDOCODE

GLOBAL DEFINITIONS

- QUEUE SampleQueue WITH SIZE 16
- BINARY SEMAPHORE SampleReady
- PERIODIC TIMER SampleTimer = 10 ms
- STRUCT Sample { sensor_id, value, timestamp }

```
BEGIN
  CREATE SampleQueue WITH SIZE 16
  CREATE BINARY SEMAPHORE SampleReady
  CREATE TASK AcquisitionTask WITH PRIORITY 3
  CREATE TASK ProcessingTask WITH PRIORITY 2
  CREATE TIMER SampleTimer PERIODIC EVERY 10 ms
  START SampleTimer
  START SCHEDULER
END

TIMER CALLBACK SampleTimerCallback
BEGIN
  GIVE SampleReady
END

ISR ADC_DATA_READY_ISR
BEGIN
  ACKNOWLEDGE ADC INTERRUPT
  SIGNAL ADC_SAMPLE_COMPLETE
END

TASK AcquisitionTask
BEGIN
  WHILE TRUE
    WAIT SampleReady
    START ADC CONVERSION
    WAIT UNTIL ADC_SAMPLE_COMPLETE
    READ ADC VALUE
    CREATE sample WITH sensor_id, value, CURRENT_TIME
    SEND sample TO SampleQueue
  ENDWHILE
END

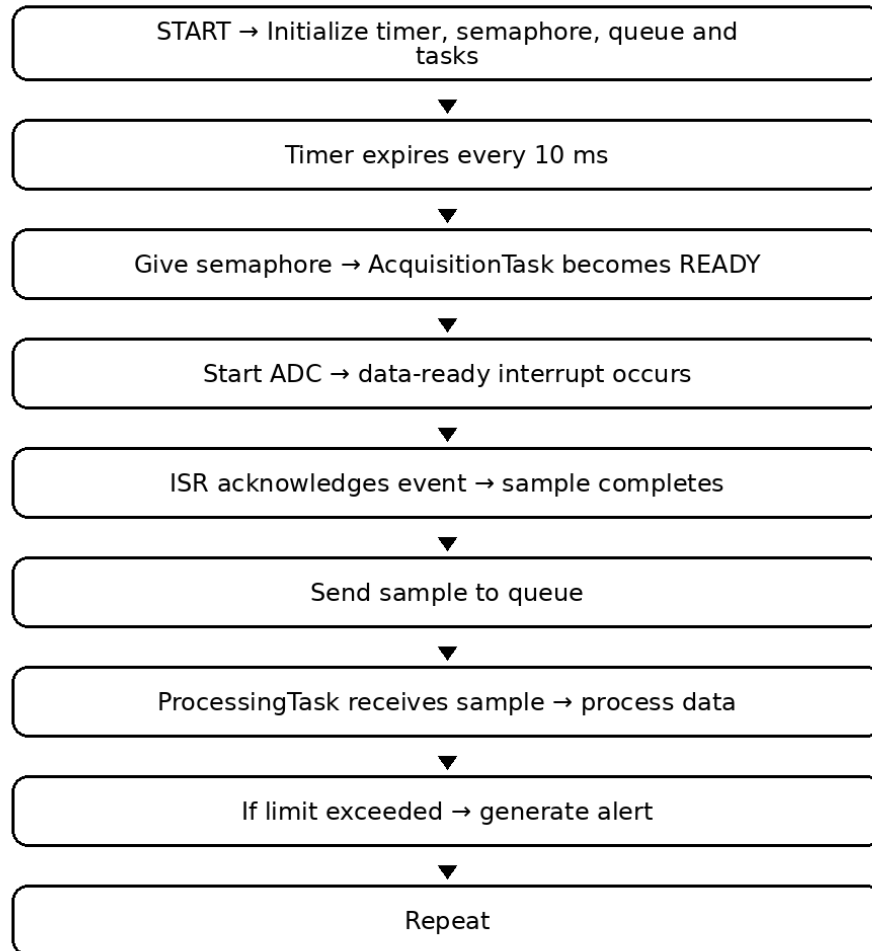
TASK ProcessingTask
BEGIN
  WHILE TRUE
    RECEIVE sample FROM SampleQueue
    PROCESS sample
    IF sample.value > LIMIT THEN
      GENERATE ALERT
    
```

```
ENDIF
ENDWHILE
END
```

4. RTOS OPERATION SUMMARY

Mechanism	Performed by	Purpose
Periodic timer	SampleTimer	Creates the fixed 10 ms sampling schedule
Binary semaphore	AcquisitionTask / timer callback	Synchronizes acquisition with the sampling event
ISR	ADC DATA READY ISR	Handles the hardware data-ready event quickly
Queue	AcquisitionTask → ProcessingTask	Buffers completed samples safely
Scheduler	RTOS kernel	Chooses the highest-priority READY task

LOGICAL FLOW DIAGRAM – Q26



5. EXPLANATION

The timer controls when acquisition should occur. The semaphore prevents continuous polling and releases the acquisition task only at the sampling event. The ISR responds quickly to the ADC event and avoids lengthy processing inside interrupt context. The queue provides safe task-to-task communication so the processing task can consume samples independently.

KEY POINTS

- Fixed periodic sampling is provided by the timer.
- Semaphore provides synchronization.
- ISR performs only short interrupt work.
- Queue provides buffered inter-task communication.
- Processing is kept outside the ISR.

ASSUMPTIONS

- ADC conversion completes within the sampling period.
- Queue capacity is sufficient for expected bursts.
- Processing can keep up with the average sample rate.
- The RTOS supports timers, semaphores, queues, tasks and ISRs.

SUMMARY

The system uses a periodic timer to control sampling, a semaphore to synchronize the acquisition task, an ISR to handle the ADC data-ready event, and a queue to transfer completed samples safely to the processing task.

REAL-WORLD EXAMPLE

An industrial motor vibration-monitoring controller can use this design to sample an accelerometer every 10 ms, respond to the ADC data-ready interrupt, queue the timestamped vibration sample, and analyze it for abnormal vibration. If the measured level crosses a configured limit, the processing task can raise a maintenance alert.

QUESTION 27

Write pseudocode for round-robin scheduling with a 10ms time slice for 3 equal-priority tasks.

ANSWER

1. PROBLEM UNDERSTANDING

Three equal-priority tasks are continuously ready to execute. The scheduler must give each task a 10 ms CPU time slice and rotate among them so that one task does not monopolize the processor.

2. RTOS MECHANISM USED

Round-robin scheduling is used for tasks at the same priority. A scheduler tick measures the 10 ms quantum. When the quantum expires, the running task is moved to the end of the ready queue and the next READY task is selected.

3. STRUCTURED PSEUDOCODE

GLOBAL DEFINITIONS

- TIME_SLICE = 10 ms
- READY_QUEUE = [TaskA, TaskB, TaskC]
- TaskA, TaskB and TaskC have equal priority
- Scheduler tick tracks the remaining quantum

```
BEGIN
  CREATE TaskA WITH PRIORITY 3
  CREATE TaskB WITH PRIORITY 3
  CREATE TaskC WITH PRIORITY 3
  SET TIME_SLICE = 10 ms
  ADD TaskA, TaskB, TaskC TO READY_QUEUE
  START SCHEDULER
END

TIMER/SCHEDULER TICK
BEGIN
  DECREMENT remaining_quantum
  IF remaining_quantum = 0 THEN
    SAVE CURRENT_TASK CONTEXT
    MOVE CURRENT_TASK TO END OF READY_QUEUE
    SELECT NEXT READY TASK
    SET remaining_quantum = 10 ms
    REQUEST CONTEXT_SWITCH
  ENDIF
END

TASK TaskA
BEGIN
  WHILE TRUE
    EXECUTE WORK_A
  ENDWHILE
END

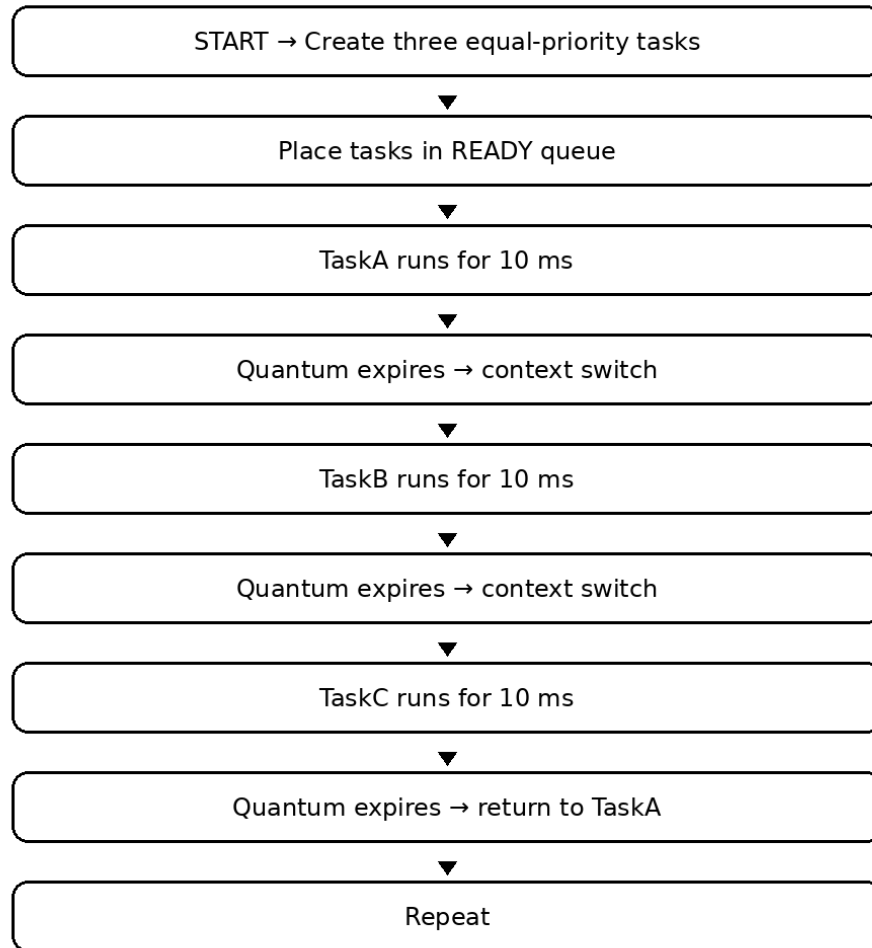
TASK TaskB
BEGIN
  WHILE TRUE
    EXECUTE WORK_B
  ENDWHILE
END

TASK TaskC
BEGIN
  WHILE TRUE
    EXECUTE WORK_C
  ENDWHILE
END
```

4. RTOS OPERATION SUMMARY

Mechanism	Performed by	Purpose
Equal priority	TaskA, TaskB, TaskC	Places all three tasks in the same scheduling class
10 ms time slice	Scheduler tick	Limits one task's CPU turn
Ready queue	RTOS scheduler	Maintains cyclic order
Context switch	RTOS kernel	Changes the running task
Round-robin rotation	Scheduler	Moves the current task to the end of the queue

LOGICAL FLOW DIAGRAM – Q27



5. EXPLANATION

All three tasks have the same priority, so the scheduler uses the time slice to share CPU time fairly. After 10 ms, the current task's context is saved and that task is placed at the end of the READY queue. The next READY task runs for the next 10 ms. The cycle continues as long as the tasks remain READY.

KEY POINTS

- Equal-priority tasks share CPU time cyclically.
- The time slice is exactly 10 ms.
- Context switching occurs when the quantum expires.
- Blocked tasks are skipped until they become READY.
- Round-robin does not change the task priorities.

ASSUMPTIONS

- The scheduler tick is accurate enough for a 10 ms quantum.

- All three tasks are eligible to run continuously in this simplified case.
- No higher-priority task is considered in the example.
- Context-switch overhead is small compared with 10 ms.

SUMMARY

Round-robin scheduling gives the three equal-priority tasks a fair 10 ms CPU time slice. When a slice expires, the current task is moved to the end of the READY queue and the next equal-priority task runs.

REAL-WORLD EXAMPLE

An automated warehouse robot controller may use equal-priority housekeeping tasks for navigation-status updates, battery monitoring, and telemetry preparation. A 10 ms round-robin slice allows each housekeeping task to receive regular CPU time without permanently favoring one of them.

QUESTION 28

Write pseudocode for rate-monotonic scheduling (RMS) in a medical device with periodic and alarm tasks.

ANSWER

1. PROBLEM UNDERSTANDING

Periodic medical-monitoring tasks have known execution periods. The scheduler must assign fixed priorities according to their periods and allow an urgent alarm task to receive immediate service when a critical event occurs.

2. RTOS MECHANISM USED

Rate-monotonic scheduling assigns a higher fixed priority to a task with a shorter period. In this design, the ECG task has a shorter period than oxygen and temperature tasks, so it receives higher RMS priority. The alarm task is event-driven and is assigned the highest priority for urgent handling.

3. STRUCTURED PSEUDOCODE

GLOBAL DEFINITIONS

- ECG_PERIOD = 20 ms
- OXYGEN_PERIOD = 50 ms
- TEMPERATURE_PERIOD = 100 ms
- Priority: Alarm > ECG > Oxygen > Temperature

```
BEGIN
  CREATE ECGTask WITH PERIOD 20 ms, PRIORITY 4
  CREATE OxygenTask WITH PERIOD 50 ms, PRIORITY 3
  CREATE TemperatureTask WITH PERIOD 100 ms, PRIORITY 2
  CREATE AlarmTask WITH PRIORITY 5
  START PERIODIC RELEASE TIMERS
  START SCHEDULER
END
```

```
TASK ECGTask
BEGIN
  WHILE TRUE
    WAIT FOR NEXT 20 ms RELEASE
    READ ECG
    PROCESS ECG
  ENDWHILE
END
```

```
TASK OxygenTask
BEGIN
  WHILE TRUE
    WAIT FOR NEXT 50 ms RELEASE
    READ OXYGEN LEVEL
    PROCESS OXYGEN
  ENDWHILE
END
```

```
TASK TemperatureTask
BEGIN
  WHILE TRUE
    WAIT FOR NEXT 100 ms RELEASE
    READ TEMPERATURE
    PROCESS TEMPERATURE
  ENDWHILE
END
```

```
ISR AlarmSensorISR
BEGIN
  ACKNOWLEDGE ALARM INTERRUPT
  SIGNAL AlarmEvent
```

```

END

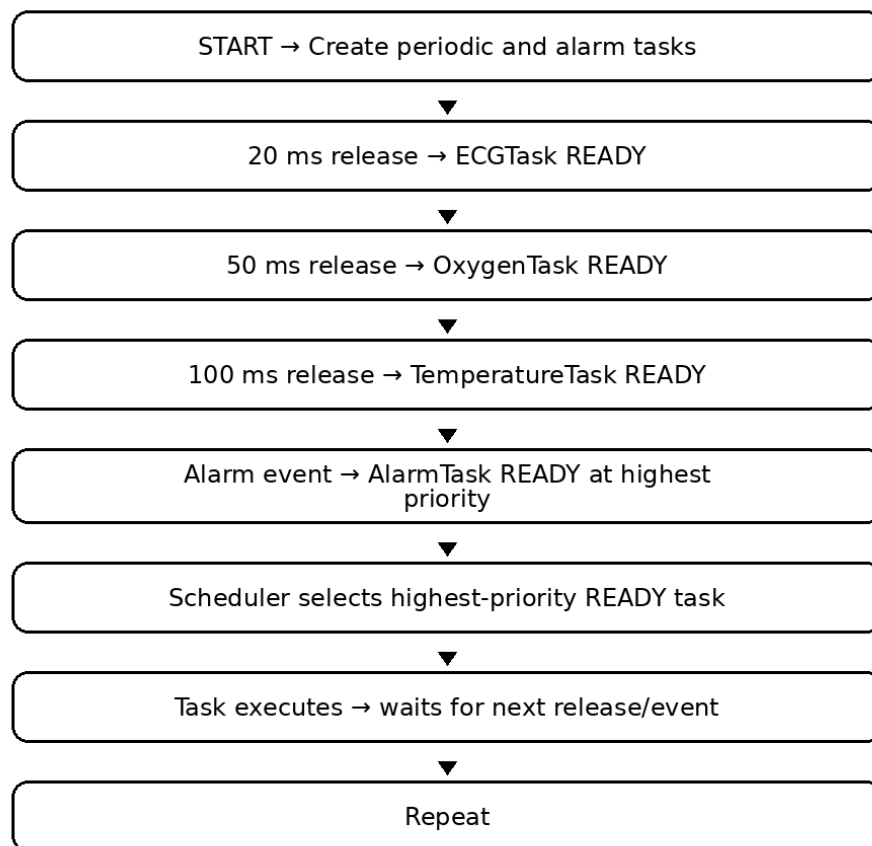
TASK AlarmTask
BEGIN
  WHILE TRUE
    WAIT AlarmEvent
    SOUND AUDIBLE ALARM
    DISPLAY EMERGENCY WARNING
  ENDWHILE
END

```

4. RTOS OPERATION SUMMARY

Mechanism	Performed by	Purpose
RMS fixed priority	RTOS scheduler	Shorter period receives higher fixed priority
Periodic release timers	ECG/Oxygen/Temperature tasks	Release tasks at their specified periods
Alarm ISR	AlarmSensorISR	Detects urgent hardware event
Alarm event	AlarmEvent	Wakes the alarm task
Preemption	RTOS kernel	Allows the highest-priority READY task to run

LOGICAL FLOW DIAGRAM – Q28



5. EXPLANATION

RMS keeps priorities fixed and derives them from task periods: the shorter the period, the higher the priority. Thus the 20 ms ECG task has higher RMS priority than the 50 ms oxygen task, which has higher priority than the 100 ms temperature task. The alarm task is event-driven and is given the highest priority in this design so a critical event can be handled immediately.

KEY POINTS

- RMS uses fixed priorities.
- Shorter period means higher RMS priority.
- Periodic tasks are released at known intervals.

- The alarm task is event-driven.
- The scheduler can preempt lower-priority work when a higher-priority task becomes READY.

ASSUMPTIONS

- The periodic task set is feasible for the processor.
- Execution times are short enough to meet deadlines.
- Alarm handling is intentionally assigned the highest priority.
- The RTOS supports fixed-priority preemptive scheduling.

SUMMARY

RMS assigns fixed priorities according to task periods: shorter-period periodic tasks receive higher priority. The alarm task is given the highest priority so an urgent event can be handled immediately.

REAL-WORLD EXAMPLE

A patient-monitoring device in an ICU can use this scheduling approach for ECG sampling, oxygen monitoring, and temperature monitoring. If a dangerous patient condition triggers an alarm sensor, the alarm task is released at the highest priority so the device can immediately activate an audible warning and emergency display.

QUESTION 29

Write pseudocode for a home automation system showing READY, RUNNING, BLOCKED, and SUSPENDED states for tasks.

ANSWER

1. PROBLEM UNDERSTANDING

The system must demonstrate the four task states. A task waits in READY until selected by the scheduler, becomes RUNNING when it executes, becomes BLOCKED while waiting for an event, and becomes SUSPENDED when the controller explicitly pauses it.

2. RTOS MECHANISM USED

Task-state management is demonstrated using scheduler selection, event-based blocking, and explicit suspension/resumption. The state is changed according to the event that occurs and the scheduler's decision.

3. STRUCTURED PSEUDOCODE

GLOBAL DEFINITIONS

- STATES = READY, RUNNING, BLOCKED, SUSPENDED
- Events: LightCommand, TemperatureUpdate, DoorAlert
- Tasks: LightingTask, ClimateTask, SecurityTask

```
BEGIN
  CREATE LightingTask WITH PRIORITY 3
  CREATE ClimateTask WITH PRIORITY 2
  CREATE SecurityTask WITH PRIORITY 4
  SET ALL TASKS TO READY
  START SCHEDULER
END
```

```
TASK LightingTask
BEGIN
  WHILE TRUE
    WAIT LightCommand
    STATE = BLOCKED
    WHEN LightCommand ARRIVES:
      STATE = READY
      CONTROL LIGHTS
  ENDWHILE
END
```

```
TASK ClimateTask
BEGIN
  WHILE TRUE
    WAIT TemperatureUpdate
    STATE = BLOCKED
    WHEN TemperatureUpdate ARRIVES:
      STATE = READY
      CONTROL HVAC
  ENDWHILE
END
```

```
TASK SecurityTask
BEGIN
  WHILE TRUE
    WAIT DoorAlert
    STATE = BLOCKED
    WHEN DoorAlert ARRIVES:
      STATE = READY
      PROCESS SECURITY EVENT
  ENDWHILE
END
```

```

CONTROLLER
BEGIN
  IF maintenance_mode = TRUE THEN
    SUSPEND LightingTask
    SUSPEND ClimateTask
  ELSE
    RESUME LightingTask
    RESUME ClimateTask
  ENDIF
END

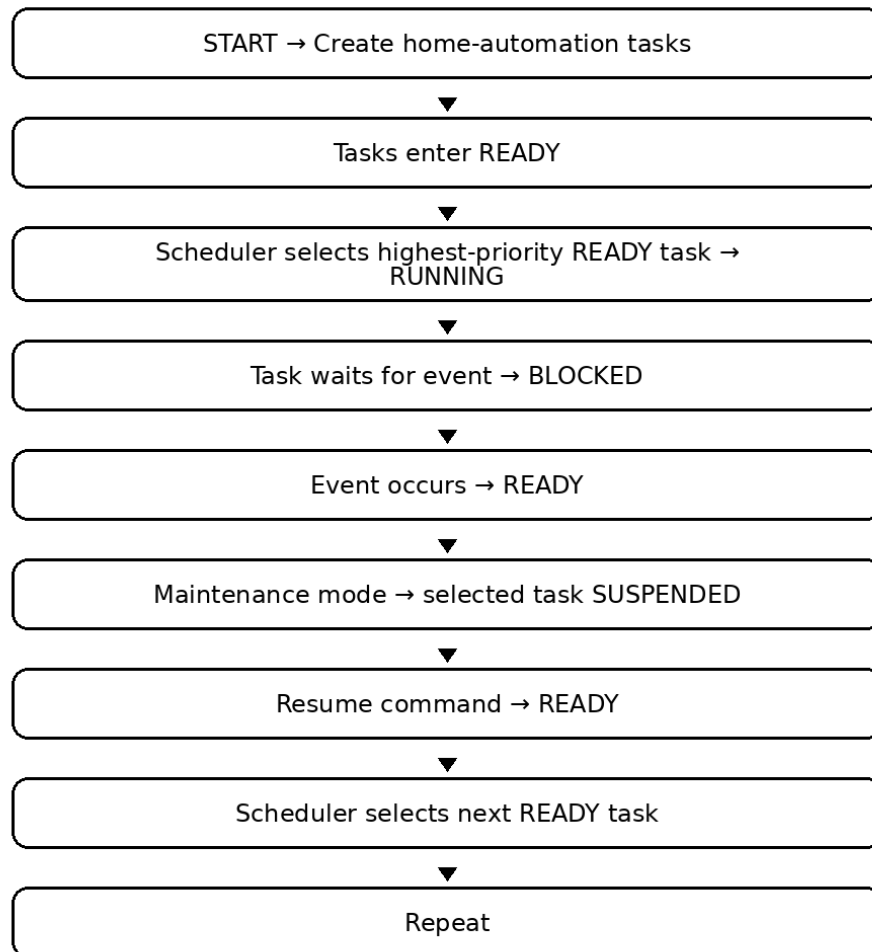
SCHEDULER
BEGIN
  SELECT highest-priority READY task
  SET selected task = RUNNING
  EXECUTE selected task
  IF task waits for event THEN
    SET task = BLOCKED
  ELSE IF task is suspended THEN
    SET task = SUSPENDED
  ELSE
    SET task = READY
  ENDIF
END

```

4. RTOS OPERATION SUMMARY

Mechanism	Performed by	Purpose
READY	RTOS scheduler	Task is eligible to run
RUNNING	RTOS scheduler	Selected task is executing
BLOCKED	Task/event mechanism	Task waits for an event or resource
SUSPENDED	Controller	Task is explicitly paused
RESUME	Controller/RTOS	Suspended task becomes eligible again

LOGICAL FLOW DIAGRAM – Q29



5. EXPLANATION

The scheduler selects a READY task and changes it to RUNNING. When the task waits for an event, it becomes BLOCKED and is not selected until that event occurs. An explicit maintenance command can move a task to SUSPENDED. When the task is resumed, it becomes eligible to run again and returns to READY.

KEY POINTS

- READY means eligible for CPU selection.
- RUNNING means currently executing.
- BLOCKED means waiting for an event/resource.
- SUSPENDED means explicitly paused.
- State transitions have clear scheduler or event triggers.

ASSUMPTIONS

- Events are delivered reliably.
- Suspended tasks remain inactive until resumed.
- Task priorities are configured before scheduling.
- The RTOS supports the required task-state operations.

SUMMARY

The home-automation tasks demonstrate READY, RUNNING, BLOCKED, and SUSPENDED states. The scheduler selects READY tasks, events release BLOCKED tasks, and explicit suspend/resume commands control SUSPENDED tasks.

REAL-WORLD EXAMPLE

A smart apartment controller can use these states for lighting, HVAC, and security tasks. For example, the lighting task can remain **BLOCKED** while waiting for a user command, become **READY** when the command arrives, run when selected by the scheduler, and be **SUSPENDED** during maintenance.

QUESTION 30

Write pseudocode for a home automation system showing task state transitions and scheduler operation.

ANSWER

1. PROBLEM UNDERSTANDING

The system must show how task state transitions interact with scheduler operation. Tasks enter READY after creation or event release, move to RUNNING when selected, become BLOCKED while waiting for events, and may become SUSPENDED through an explicit controller command.

2. RTOS MECHANISM USED

A preemptive RTOS scheduler repeatedly selects the highest-priority READY task. Event mechanisms cause BLOCKED tasks to return to READY, while explicit suspend/resume operations move tasks between SUSPENDED and READY.

3. STRUCTURED PSEUDOCODE

GLOBAL DEFINITIONS

- STATES = READY, RUNNING, BLOCKED, SUSPENDED
- Events: OccupancyEvent, PowerUpdate, UserCommand
- READY_LIST stores tasks eligible to execute

```
BEGIN
  CREATE OccupancyTask WITH PRIORITY 4
  CREATE LightingTask WITH PRIORITY 3
  CREATE EnergyTask WITH PRIORITY 2
  CREATE ControllerTask WITH PRIORITY 5
  ADD ACTIVE TASKS TO READY_LIST
  START SCHEDULER
END
```

```
SCHEDULER
BEGIN
  LOOP FOREVER
    SELECT highest-priority task FROM READY_LIST
    SET selected task = RUNNING
    RUN selected task
    IF task waits for event THEN
      SET task = BLOCKED
    ELSE IF task is suspended THEN
      SET task = SUSPENDED
    ELSE
      SET task = READY
      ADD task TO READY_LIST
    ENDIF
  ENDLOOP
END
```

```
TASK OccupancyTask
BEGIN
  WHILE TRUE
    WAIT OccupancyEvent
    DETECT OCCUPANCY
    IF person_detected THEN
      SIGNAL UserCommand
    ENDIF
  ENDWHILE
END
```

```
TASK LightingTask
BEGIN
  WHILE TRUE
    WAIT UserCommand
```

```

CONTROL LIGHTING
ENDWHILE
END

TASK EnergyTask
BEGIN
WHILE TRUE
    WAIT PowerUpdate
    CALCULATE ENERGY USAGE
    APPLY ENERGY-SAVING POLICY
ENDWHILE
END

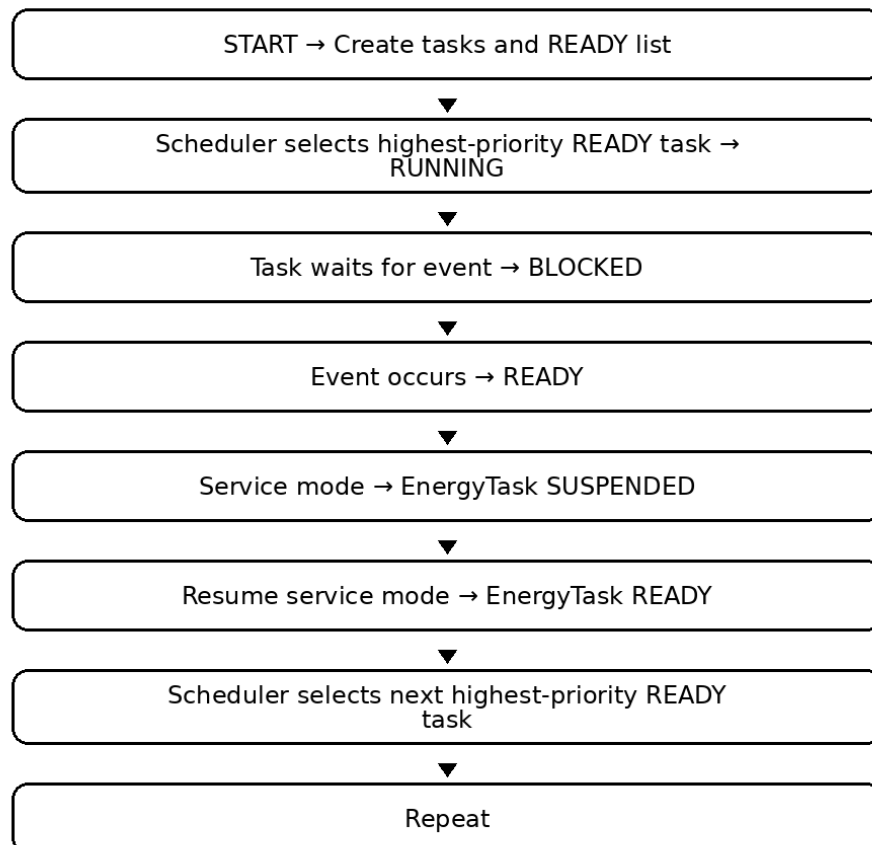
TASK ControllerTask
BEGIN
IF service_mode = TRUE THEN
    SUSPEND EnergyTask
ELSE
    RESUME EnergyTask
ENDIF
END

```

4. RTOS OPERATION SUMMARY

Mechanism	Performed by	Purpose
Task creation	RTOS	Creates tasks and places active tasks in READY
Scheduler	RTOS kernel	Selects the highest-priority READY task
RUNNING	Scheduler	Represents the selected executing task
BLOCKED	Event mechanism	Task waits without using CPU
SUSPENDED	Controller	Task is explicitly stopped
READY	Event/resume	Task becomes eligible to run again

LOGICAL FLOW DIAGRAM – Q30



5. EXPLANATION

The scheduler and task-state transitions work together continuously. A READY task is selected and becomes RUNNING. If it waits for an event, it becomes BLOCKED and leaves the set of runnable tasks. An explicit service command can suspend a task, placing it in SUSPENDED until resume. When an event occurs or a suspended task is resumed, it returns to READY and can be selected again according to priority.

KEY POINTS

- The READY list contains runnable tasks.
- The scheduler selects the highest-priority READY task.
- BLOCKED tasks wait for events and do not consume CPU.
- SUSPENDED tasks require explicit resumption.
- Every state transition has a defined trigger.

ASSUMPTIONS

- The scheduler is preemptive priority-based.
- Events are reliably delivered.
- Suspension is controlled by the controller task.
- Task execution times are suitable for the home-automation response requirement.

SUMMARY

The scheduler repeatedly selects the highest-priority READY task and manages its transitions to RUNNING, BLOCKED, SUSPENDED, and back to READY according to events and controller commands.

REAL-WORLD EXAMPLE

A smart-home energy-management controller can use this design to coordinate occupancy detection, lighting control, and energy optimization. Occupancy events can wake the occupancy task, lighting can respond to user commands, and an energy task can be suspended during service mode and resumed afterward.